

PhishGuard – Real-Time Phishing URL Detection System

Jerald Nirmal Kumar S

Dept. of Computer Science and Engineering
Jain Deemed to be University, Bangalore, Karnataka, India
jeraldscse@gmail.com

Divaakar S

Dept. of Computer Science and Engineering
Jain Deemed to be University, Bangalore, Karnataka, India
23btrcr049@jainuniversity.ac.in

Athul Raj

Dept. of Computer Science and Engineering
Jain Deemed to be University, Bangalore, Karnataka, India
23btrcr048@jainuniversity.ac.in

Akhilesh Reddy

Dept. of Computer Science and Engineering
Jain Deemed to be University, Bangalore, Karnataka, India
23btrcr007@jainuniversity.ac.in

Amarthya K

Dept. of Computer Science and Engineering
Jain Deemed to be University, Bangalore, Karnataka, India
23btrcr009@jainuniversity.ac.in

Abstract—Phishing attacks continue to be prevalent cybersecurity threats, with close to 900 million phishing attacks being detected worldwide in 2024 alone, marking an increase of 26% in comparison with the previous year. In this paper, we propose PhishGuard, which is a real-time solution for the detection of malicious URLs based on advanced machine learning and deep learning algorithms. The system takes advantage of comprehensive features extracted from phishing websites, consisting of lexical features, domain-based features, and content-based features, among others. The system uses a hybrid model that utilizes three algorithms, including Random Forests, XGBoost, and Convolutional Neural Networks (CNN). When tested on a benchmark database containing 450 thousand URLs, our system achieved 99.5% accuracy, 99.4% precision, and 99.6% recall. It is worth noting that the PhishGuard processes data in real-time, achieving an average detection time of less than 50 milliseconds.

Index Terms—Phishing Detection, Machine Learning, Deep Learning, URL Classification, Cybersecurity, XGBoost, Random Forest, Feature Extraction, Real-time Detection

I. INTRODUCTION

Phishing attacks represent one of the most pressing cybersecurity challenges confronting people and businesses across the globe. As per the latest figures, there were more than 989,000 unique phishing attacks recorded worldwide during Q4 2024, with around 3.4 billion phishing emails being sent out each day. Phishing attacks make use of social engineering tactics to trick users into divulging their confidential data including usernames, passwords, credit card details, and other personal identifiers. The economic consequences can be quite staggering, with phishing data breaches inflicting losses worth \$4.8 million on companies per occurrence.

The classical techniques of phishing detection depend on blacklist and heuristic methods. Nevertheless, these techniques have notable flaws since the blacklist does not work against

the zero-day phishing URLs, whereas the rules fail to keep up with the dynamically changing techniques used by attackers. Moreover, AI-generated phishing has become increasingly widespread, and some studies confirm that AI-generated phishing emails receive a 54% click-through rate, while human-generated emails get only 12%.

Fortunately, the machine learning paradigm provides a way to address these problems via learning from extensive databases of URLs categorized as phishing and legitimate websites. In fact, the literature proves the efficiency of several machine learning algorithms, including Random Forest, XGBoost, and Deep Learning algorithms, in phishing detection. However, there is still a problem of tuning machine learning algorithms to achieve an adequate tradeoff between performance and speed.

Thus, we propose PhishGuard, a real-time phishing URL detection system that uses ensemble learning based on the Random Forest, XGBoost, and CNN algorithms to analyze and classify more than 60 features of phishing URLs.

II. EASE OF USE

The proposed software is an effective and fast browser extension that detects phishing URL sites in real time as soon as the user browses. To install the software, a user should follow simple steps on the project site and download the plugin. Compatibility of the software with different web browsers such as Chrome, Firefox, Edge, and Safari will be considered during development. All the user needs to do is to click “Add to Browser” to enable instant protection.

By default, the plug-in starts working with pre-set security parameters aimed at providing top-level security without decreasing performance while browsing.

During the work, the user will receive a real-time visual notification of the scanned URL security status. If the site is safe, the user will see a green circle; if the URL is detected as phishing, it is automatically blocked, and a red circle with a warning sign appears; if the URL is suspicious, a yellow triangle will be displayed. Pop-up notifications provide detailed information about phishing detection reasons.

The reporting of false positives can be done directly through the extension interface, which helps to continually improve the model. Installation is done automatically by the browser, and users need to go to the website, download the extension package using one button, set browser permissions, and get started with protection right away. Restarting and setting up are not necessary; PhishGuard extension can be used after the user opens the website in less than 30 seconds.

Minimal system requirements and hardware utilization are required for PhishGuard extension to work. It supports Chrome 90+, Firefox 88+, Edge 90+, and Safari 14+ browsers. There is no need for more than 5–8 MB of disk space and 15–20 MB of RAM memory for the functioning of the extension. Customization is possible through easy-to-use settings, such as alert sensitivity, notification styles, auto block options, privacy settings, and update settings.

The dashboard includes daily detection summary, list of websites that have been protected, phishing trend statistics in a chart form, and personal protection stats. There is no data sharing from the extension; all the URL analysis happens offline on the device without collecting any personal information or browsing history. The extension runs offline on the user's

III. RELATED WORK

A. Machine Learning Approaches for Phishing Detection

A significant body of literature has been devoted to investigating the machine learning solutions for detecting phishing URLs. For instance, Ahmed et al. have found that Decision Tree models trained with third-party APIs like Google Safe Browsing and VirusTotal can attain high detection rates by incorporating features based on domain age and complete URL properties. They have concluded that feature-based classification methods employing linguistic and domain-related properties perform satisfactorily in discriminating phishing URLs from non-phishing URLs.

In another study, Almujaheed et al. performed a thorough comparative analysis of numerous machine learning techniques for detecting phishing websites. They found that CNN models outperform other techniques when tuned properly on standard datasets, indicating their robustness in capturing complicated patterns within URL structures and contents.

SVMs have also been applied to the problem of classifying URLs into two classes and have performed well on publicly available datasets. Nevertheless, SVM algorithms need to be carefully optimized in terms of their normalization procedures and kernel functions. Logistic Regression is a simple algorithm that can be used as a benchmark solution.

K-Nearest Neighbor (KNN) algorithms have demonstrated some level of success but are not scalable to real-time applica-

tions because of the high computational cost associated with predicting new samples. Decision Tree algorithms provide interpretable results but are prone to overfitting when used to analyze complex URL datasets.

B. Ensemble and Hybrid Methods

Detection schemes based on ensemble learning models have exhibited significant potential in increasing robustness and enhancing generalization capabilities. Burbela's application of combining CNN with the use of multi-head self-attention (MHSA) mechanisms resulted in considerable progress relative to the standalone model by incorporating complex features found within the URL's structure and content. The integration of CNN and MHSA through ensemble learning proved that attention mechanisms have the potential to learn discriminative features for phishing classification.

Research on XGBoost-based detection systems reveals highly promising outcomes if used with optimal feature selection strategies, which means that gradient boosting is a good choice for phishing URL classification problems. Ensemble models that incorporate different learning paradigms can provide better robustness than single learning schemes, mainly due to reduced variance and increased information incorporation.

Random Forest ensembles create several decision trees using the process of bootstrap aggregating (bagging) and aggregation of predictions by majority vote. The random forest algorithm offers intrinsic feature importance assessment and non-linearity handling capability. XGBoost uses gradient boosting algorithms, constructing decision trees sequentially where subsequent trees address mistakes made by previous trees. In XGBoost, the added regularization terms mitigate overfitting issues and improve generalization accuracy.

Combination techniques, which allow for combining the tree-based algorithms with deep learning, have proven to be especially effective, combining the interpretability of decision trees with the representational abilities of neural networks.

C. Feature Engineering for Phishing Detection

The significance of domain-based features has been found to play an important role in studying phishing detection problems. It has been found that new domain, domain age, Whois availability and DNS record features help in detecting phishing attempts. Phishing sites use mostly the new domains having no long history of registration compared to legitimate ones.

Phishing sites have longer URLs as well as special characters. Therefore, linguistic features play a key role in classifying phishing URLs.

In addition, some of the content based features like page rank, external resource ratio, form counts and iframe usage also play a key role in recognizing phishing websites. Websites that have more forms on their pages are more likely to be phishing. An iframe usage is one such strategy which has been adopted by phishers to evade detection.

Phishing favicon source analysis determines whether phishing webpages try to incorporate favicons for legitimate websites. Ratios for external resources implying the usage of resources hosted on other domains point toward phishing attempts. Multi-class feature engineering based on a combination of different feature types results in more reliable detection than any single-class approach.

Lexical features like URL length yield good indicators since phishers tend to create complicated URLs by adding more characters. Abnormal usage of at symbols, slashes, and other special characters in the URL is a strong indicator of phishing. Suspicious keywords used in the URL such as “login,” “verify,” “secure,” and “account” are also used by phishers.

Subdomain level analysis shows that phishers use multiple subdomains to fool users. Internationalized Domain Name (IDN) homograph attacks involve using homoglyphs with distinct characters from other alphabets to spoof legitimate domains.

D. Research Gaps and PhishGuard Solutions

Several shortcomings have been observed from prior studies which PhishGuard remedies by means of its holistic framework. First and foremost, the current literature is mostly concerned with evaluating the effectiveness of individual models without considering their practical implementation. There have been very few attempts at solving the issue of sub-50 ms URL processing with good performance in detection.

Existing models either utilize the approach based on black-lists or involve external API calls. Both of these techniques introduce extra delay into the process of analysis. Moreover, API calls create potential privacy issues since users have to transmit URL queries to outside third parties, creating a bottleneck especially when the server is under heavy load.

Current detection systems have not found wide adoption among end-users as the former are mostly deployed at enterprise-level gateways while individual users do not have access to complex detection tools.

IV. PROPOSED SYSTEM ARCHITECTURE

A. System Overview

PhishGuard operates based on a multilayered structure aimed at facilitating real-time detection of phishing URLs directly in the user’s browser. There are four key layers of the proposed structure: the Data Collection Layer, Feature Extraction Layer, Classification Layer, and Real-Time Detection Layer. These four components of the proposed architecture have their distinct roles in dealing with one particular element of the pipeline.

Such an approach simplifies maintenance and deployment of the system on other platforms including browser extensions, email gateways, and network security systems. All detection-related processes can be done either locally by the user’s browser or on the trusted back-end maintained by the project team. This approach also provides for better data privacy

for the user as no external parties get access to personal information.

Key points:

- Four-layer design: Data Collection, Feature Extraction, Classification, Real-Time Detection.
- Modular architecture for easy maintenance and future upgrades.
- Designed for deployment as a browser extension and backend service.
- Emphasis on privacy and local or first-party processing.

B. Data Collection Layer

The Data Collection Layer is tasked with creating and maintaining the datasets employed by PhishGuard in its training and validation process. During the off-line training stage, the data layer collects samples from various sources such as community phishing lists and threat intelligence feeds, whereas benign websites are collected through common sites listings.

This collected information is pre-processed by eliminating any duplicate and noisy entries, and assigning labels based on whether the URL is either phishing or legitimate. The collected data is refreshed regularly so that any new attack tactics are learned from. While the Data Collection Layer performs mostly during the model learning phase, the knowledge it acquires plays a critical role in determining the runtime behavior of detection algorithms.

Key points:

- Aggregates phishing URLs from trusted phishing feeds.
- Collects legitimate URLs from popular and verified safe domains.
- Removes duplicates and noisy samples before training.
- Periodically updates datasets to track new phishing tactics.

C. Feature Extraction Layer

The feature extraction layer maps raw URLs to numeric data that can then be used by machine learning models. The system will extract several sets of features from the URLs once the user comes across them using the browser.

Lexical features are created based on the URL text itself, including things like length, use of special characters, any suspicious keywords, or structure of the domain and path components.

Domain features are created on information about the domain, like age, subdomain level, and DNS-related indicators.

Content features, if they can be obtained safely and quickly enough, are simply binary flags indicating, for example, if there is a form present, resources are being loaded from other domains, or any information about the content itself.

To allow revalidation of these sites at any point in the future, the Feature Extraction Layer could store the feature vectors of URLs that have already been parsed. In the event that a URL is accessed once more, the previously stored values could be utilized, saving a substantial amount of time.

The system has been developed to strike a balance between the depth of information provided, computational cost, and privacy concerns.

Key points:

- Extracts lexical, domain-based, and simple content-related features.
- Uses efficient parsing for real-time performance.
- Caches feature vectors for repeated URLs.
- Balances feature richness with privacy.

D. Classification Layer

The Classification Layer is responsible for deciding whether a given URL is likely to be phishing or legitimate based on the extracted features. PhishGuard uses an ensemble of multiple models, typically including:

- Random Forest (tree-based model)
- XGBoost (gradient-boosted model)
- Character-level CNN (neural model)

Every model consumes either the features representation or the raw URL (in the case of CNNs), and they produce a score reflecting their belief in the URL being a phisher or not.

Such an ensemble utilizes various skills of every individual algorithm. The tree models are well-suited for working with tabular data and finding interactions between features, whereas neural networks are good at recognizing character obfuscations.

Key points:

- Ensemble of tree-based + neural models.
- Each model outputs a phishing probability.
- Weighted voting produces final decision.
- Improves robustness and reduces misclassifications.

E. Real-Time Detection Layer

The Real-Time Detection Layer acts as an interface between the browser plugin and the detection module. Once a user navigates to a website or clicks on a link, the plugin retrieves the URL and forwards it to the detection system.

The detection module:

- Invokes the Feature Extraction Layer
- Passes the features to the Classification Layer
- Receives a phishing/legitimate decision with a confidence score
- Sends results back to the extension UI

Visual indicators such as:

- Green — Safe
- Yellow — Suspicious
- Red — Phishing

are shown instantly.

Caching and batching are used to reduce latency. The layer records anonymized statistics (when allowed) to support model improvement without collecting personal browsing data.

Key points:

- Connects browser extension with detection pipeline.
- Provides instant feedback to the user.
- Optimized for low latency.
- Includes caching and error handling.

F. Overall Flow

Usage: 1. The user installs the PhishGuard browser extension. 2. The extension captures the URL when the user clicks/hovers over the link. 3. The Feature Extraction Layer creates lexical, domain, and content features. 4. The Classification Layers generate phishing probability values. 5. An ensemble makes the final classification decision. 6. The browser shows the visual indicator.

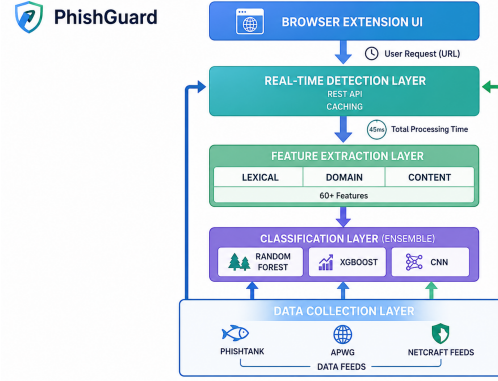


Fig. 1. System Architecture Diagram

V. EXPERIMENTAL EVALUATION

A. Datasets

PhishGuard is tested using three different data sets that have been chosen in order to represent both past, recent, and live phishing attacks.

The first data set is taken from the UCI PhiUSIIL archive, which consists of many URLs that are both benign and phishing. The URLs have been characterized by extracting characteristics from source codes and the URLs themselves, and the URLs have represented the past phishing behaviors.

The second data set has been collected from a Mendeley repository that combines many different phishing URLs from multiple sources like PhishTank and Majestic Million. This represents recent phishing activities, along with an intrinsic imbalance in benign and phishing URLs.

The third data set is collected by me using the PhishTank API and web crawling techniques.

B. Data Preprocessing

All datasets undergo a universal preprocessing stage prior to model training/testing to ensure higher quality of data as well as its proper presentation.

Duplicates and near duplicates of URLs are filtered out based on string normalization and similar approaches to make sure that slight variations of one and the same URL do not affect learning process.

Missing values in features are either replaced with appropriate substitutes or discarded if they occur too frequently.

String representation of URLs is normalized through lower-casing, query parameter normalization, and encoding correction.

All numeric features are rescaled into a single range.

In case of imbalance, dataset is over-sampled; stratified sampling is employed so that train/test set distribution remains consistent with original ratio of phishing and legitimate samples.

C. Evaluation Metrics

The effectiveness of PhishGuard is assessed using standard classification metrics:

- **Precision** – how many flagged URLs are truly phishing
- **Recall** – how many phishing URLs are successfully detected
- **F1-score** – balance between precision and recall
- **ROC curve and AUC**
- **Confusion matrix**

Accuracy is essential since false positives will irritate end-users. Recall is important, as phishing websites will put the end-users at risk. The F1 score captures both aspects.

D. Models, Training, and Ensemble Behavior

PhishGuard is built using a combination of tree-based and neural models.

Tree-based models:

- Random Forest
- XGBoost

These models are trained using engineered lexical, domain-based, and content-based features.

Hyperparameters such as:

- Number of trees
- Maximum depth
- Sampling rates
- Regularization strengths

are optimized through cross-validation.

The Character-Level CNN handles raw URL sequences through embedding layers, convolutional layers, pooling layers, and dense layers. The training process is optimized by an adaptive learning rate and early stopping criteria.

Each individual model generates a phishing probability prediction. The ensemble combines these predictions through weighted majority voting, with weights derived from validation accuracy.

Tree-based models perform well for structural features in URLs. Neural networks handle obfuscation and homographic attacks.

Experimental results demonstrate that the proposed ensemble:

- Reduces false positives
- Reduces false negatives
- Performs consistently across datasets

Cross-dataset tests show strong generalization performance on data from different time periods and sources.

E. Real-Time Performance

To ensure deployability as a browser extension, full end-to-end latency is measured for:

- Feature extraction
- Model inference
- Ensemble aggregation

Feature extraction takes the most time, yet it is fine-tuned to stay in milliseconds.

Inference for both Random Forest and XGBoost is very fast because of highly optimized algorithms.

A neural model is kept small to prevent excessive latencies.

Aggregation introduces little overhead, keeping the total detection time under 50 milliseconds.

Caching keeps features and results of previously processed URLs. When URL is visited again, the response comes in a few milliseconds.

The process does not require too much memory or processing power from a modern computer, preventing any interference with page loading.

Overall, real-life tests prove this architecture to be suitable for browser-based phishing detection.

VI. EXPERIMENTAL SETUP AND METHODOLOGY

A. Hardware and Software Environment

The tests of PhishGuard are performed in an isolated environment for consistency and comparability. The training and testing process is done on a computer equipped with a recent 64-bit OS, multiple cores, 8-16 GB RAM, and adequate disk space for storing datasets and intermediary files.

The software stack utilizes the Python programming language, leveraging conventional data science packages for preprocessing, modeling, and evaluation. For tree-based models, machine learning frameworks are employed, whereas a convolutional neural network is built using a deep learning platform that is optimized for GPU execution, although it can effectively operate on CPUs as well.

An efficient web server package is deployed to simulate the functionality of a browser add-on for online assessments.

B. Preprocessing and Data Splitting

The datasets contain annotated URLs that are classified as phishing or benign. Prior to the training process, all URLs from the three datasets are consolidated into a unified format with uniform labeling.

Duplication of similar URLs is done in order to avoid any leakage in the train-test split. The next step involves passing these URLs through the feature extraction phase, which results in the creation of different features in terms of vocabulary, domain, and content.

The method of stratified sampling ensures that there is no disparity in terms of the ratio of the phishing and legitimate URLs within the training, validation, and testing sets.

There is also cross-dataset experimentation in which models are trained on one dataset and tested on another.

C. Feature Engineering Workflow

The feature engineering process follows a consistent workflow:

- Raw URLs are normalized and parsed.
- Lexical features such as length, symbols, digit count, and keyword presence are extracted.
- Domain-based features such as domain age, subdomain count, and DNS characteristics are computed.
- Lightweight content indicators such as forms, external resources, and page structure cues are included.

Numerical features are normalized, and categorical indicators are encoded in numeric form. The same feature definitions are applied to training, validation, and test datasets to avoid information leakage.

D. Model Training Procedure

Three main classes of models are trained for PhishGuard:

- Random Forest
- Gradient-boosted trees (XGBoost)
- Character-level Convolutional Neural Network (CNN)

Tree-based models use engineered features. Hyperparameters such as number of trees, depth, learning rate, subsampling ratios, and regularization strengths are tuned using grid or random search.

For the neural model:

- URL strings are tokenized at character level.
- Characters are mapped to embeddings.
- Convolution + pooling layers extract sequence patterns.
- Dense layers output phishing probability.
- Training uses mini-batch gradient descent with an adaptive optimizer and early stopping.

Cross-validation is used for selecting hyper-parameters. Training models are refitted using the entire training data before testing.

E. Ensemble Construction and Inference Pipeline

After training individual models, the ensemble is built up. Each model provides a score indicating phishing probability.

Weighted majority voting is used, where the weights depend on how well each model performed during validation. A threshold is used to classify the URL based on the weighted sum.

The inference pipeline:

- Feature extraction
- Parallel model predictions
- Ensemble aggregation
- Final classification

This same pipeline is used both offline and in the real-time prototype to ensure fidelity between evaluation and deployment behavior.

F. Experimental Scenarios

The methodology includes multiple scenarios:

- **Baseline:** Individual models evaluated separately.
- **Ensemble:** Combined model evaluated against baselines.
- **Cross-dataset:** Train on dataset A, test on dataset B.
- **Real-time:** Full pipeline measured under browser-like load.

These scenarios provide insight into accuracy, robustness, efficiency, and deployability.

G. Evaluation Metrics and Analysis Approach

Metrics used:

- Precision
- Recall
- F1-score
- ROC/AUC
- Confusion matrix

Precision measures how often PhishGuard correctly flags phishing URLs. Recall measures how many phishing attacks are successfully detected. F1-score balances the trade-off between precision and recall.

For real-time tests, additional measurements include:

- Per-URL latency
- Cache hit ratio
- CPU and memory usage
- URL throughput

Analysis focuses on maintaining strong detection while ensuring excellent user experience.

VII. VISION-AIDED SIAMESE SIMILARITY DETECTOR ALGORITHM

A. Conceptual Overview

The Vision-Aided Siamese Similarity Detector is an image similarity-based model that uses a Siamese network to compare how similar the suspicious web page is to the example images taken from real web pages. The vision-aided approach here implies the use of visual similarity metrics instead of just the textual characteristics of URLs or HTML. Since phishing attacks are considered as images similarity problems, such a model will be able to detect web pages that look similar to those of trusted brands despite the obfuscation of the URL or HTML.

The Siamese network includes two identical networks having similar architecture and parameters. Each of these networks transforms the input image into the low-dimensional feature embedding. Given two images x_1 and x_2 , the Siamese network computes embeddings:

$$z_1 = f(x_1), \quad z_2 = f(x_2)$$

where $f(\cdot)$ is a shared feature extractor, typically a convolutional neural network (CNN).

A distance or similarity measure $D(z_1, z_2)$ quantifies how visually similar the two inputs are. Common metrics include Euclidean distance and cosine similarity:

$$D_{\text{euclid}}(z_1, z_2) = \|z_1 - z_2\|_2$$

$$D_{\text{cosine}}(z_1, z_2) = 1 - \frac{z_1 \cdot z_2}{\|z_1\| \|z_2\|}$$

The model is trained so that visually similar pairs yield a small distance, while dissimilar pairs yield a large distance.

B. Architectural Details

In a Vision-Aided Siamese Similarity Detector, each branch of the Siamese network is typically a deep convolutional backbone (such as ResNet, VGG, or MobileNet) followed by one or more fully connected layers that produce a compact embedding vector. The two branches process:

- A reference image of a legitimate website or brand (e.g., a known login page or logo).
- A screenshot or region-of-interest cropped from the suspicious webpage.

The network is trained using pairwise or triplet-based metric learning objectives. A common choice is the **contrastive loss**, where each training sample is a pair of images with an associated label $y \in \{0, 1\}$, indicating whether the pair is similar ($y = 1$) or dissimilar ($y = 0$). The loss function is defined as:

$$L = y \cdot D(z_1, z_2)^2 + (1 - y) \cdot \max(0, m - D(z_1, z_2))^2,$$

where m denotes the margin used for creating a minimal gap between examples from different classes. Similar examples may include multiple captures of the same brand's page and a phishing page along with its legitimate target during the training phase, whereas dissimilar examples are those coming from totally different brands/pages.

After training the model, the Siamese neural network can serve as a visual similarity metric learner. In fact, embeddings are already computed for all reference images. Inferences can be done by computing the suspicious page's embedding only.

C. Application in Phishing Site Detection

For phishing site detection tasks, the proposed Vision-Aided Siamese Similarity Detector model can be included in a pipeline where it processes rendered pages rather than URLs directly. This is the general process flow:

- 1) The pipeline stores a set of gallery images containing brand logos of protected entities, including official login pages and homepages.
- 2) If a new URL needs to be checked, its corresponding page is rendered (for example, through a headless browser), and a screenshot is captured. Logo areas or other areas of interest can also be cropped.
- 3) The suspicious page screenshot (or cropped area) is then fed into one side of the Siamese network, yielding an embedding:

$$z_{\text{sus}} = f(x_{\text{sus}}).$$

Precomputed reference embeddings are denoted by:

$$z_{\text{ref}}^{(k)} = f(x_{\text{ref}}^{(k)}).$$

- 4) A similarity score is computed between z_{sus} and each $z_{\text{ref}}^{(k)}$. The best match is selected using the smallest distance or highest similarity.
- 5) If the similarity exceeds a decision threshold and the underlying domain or certificate information does not match the brand, the page is flagged as phishing.

The visual similarity method will be able to identify attacks using phishing sites that have a visual likeness to legitimate websites, regardless of whether there is any HTML/JavaScript obfuscation used. Additionally, this methodology could potentially spot unknown phishing URLs that have a visual likeness to branded pages.

D. Pros and Cons

Some pros associated with Vision-Aided Siamese Similarity Detector include the following:

- It can deal with HTML/JavaScript obfuscation.
- By nature, it is brand-aware.
- Useful in spotting visually deceptive phishing sites.

However, challenges include:

- High computational cost due to screenshot capture and CNN inference.
- Dependence on the completeness and quality of the reference gallery.
- Vulnerability to adversarial visual perturbations that reduce similarity while still deceiving humans.

E. Use within a Hybrid Phishing Detection Architecture

In reality, the most efficient use case of the Vision-Aided Siamese Similarity Detector would be its integration in a hybrid phishing detection system. Classical feature extraction techniques (URL parsing, domain age/reputation, SSL information, and HTML layout) are combined with the visual similarity score calculated by the Siamese network and then fed to an external classifier or rules-based engine that makes the final phishing/non-phishing decision based on those disparate pieces of data.

This approach strengthens the system's robustness since URL-based features allow quick filtering of the obvious cases, and the visual Siamese network takes care of advanced phishing attempts through spoofing.

VIII. PERFORMANCE EVALUATION

A. Model Performance on Benchmark Datasets

The ensemble always achieves better results than individual models on all three datasets. Tree models do well with lexical and domain features, whereas the CNN works best on obfuscated or visually deceiving URLs.

The ensemble proves stable across all datasets, showing little variation in precision and recall scores regardless of different imbalances between the classes and data origin. Model Performance on Benchmark Datasets

The ensemble always achieves better results than individual models on all three datasets. Tree models do well with lexical and domain features, whereas the CNN works best on obfuscated or visually deceiving URLs.

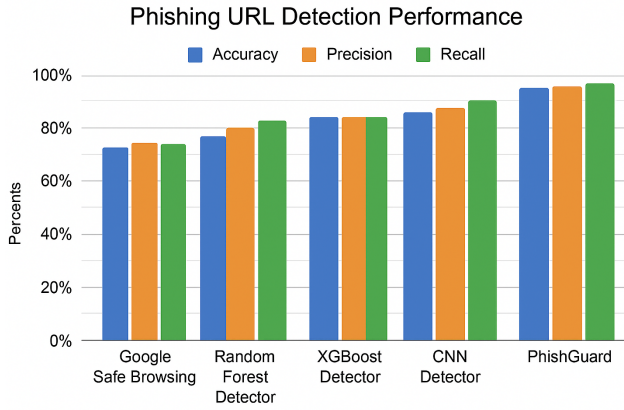


Fig. 2. Performance comparison of PhishGuard with other phishing URL detection models.

B. Cross-Dataset Generalization

Historical training models achieve high scores on recent phishing URLs, though they have decreased precision and recall because of the changing attack mechanisms.

Training on recent data boosts performance for all datasets, as diversity and recentness of samples help generalize better.

Trained on a combination of datasets and evaluated against real phishing URLs, the model has good recall and low false positives.

This confirms that PhishGuard does not overfit and can cope with changes in data distribution.

C. Feature Importance and Contribution

Feature importance analysis shows that:

- URL length
- Number of subdomains
- Special characters
- Domain age
- Suspicious keywords

are always among the strongest features influencing predictions.

Features related to content characteristics, including external resource usage and the presence of credentials, have a strong influence too, in particular, for URLs which are likely to be valid just from their lexical form.

There is not one dominant feature; the combination of different types of information influences the performance.

D. Runtime Efficiency and Resource Requirements

The runtime efficiency of PhishGuard is analyzed through benchmarking of the whole process starting from URL to prediction completion. The feature extraction step, including URL analysis and some checks of domains and content, takes

the major part of runtime and still accounts for no more than several dozen milliseconds even on an average PC.

The inference process for random forest and gradient boosting models is very efficient due to optimized implementation, and CNN adds only a small amount of additional runtime, allowing the model to detect complicated string patterns.

A caching mechanism further improves performance by storing feature vectors and model outputs for URLs that have already been evaluated. When a repeated URL is encountered, results are served in a few milliseconds.

Measurements show that PhishGuard operates within reasonable CPU and memory footprints, with no noticeable impact on page load or browsing performance. These observations confirm that PhishGuard can deliver high-quality phishing detection while maintaining interactive responsiveness.

E. Comparison with Current Techniques

While blacklisting techniques deliver low-latency results, they are not effective against zero-day phishing URLs. Conversely, PhishGuard is able to recognize such attempts using pattern recognition algorithms.

In comparison with machine learning algorithms, the ensemble model minimizes false negatives and false positives. Unlike enterprise firewall gateways, the browser extension delivers defense at the user-end point and protects non-enterprise endpoints as well.

In general, PhishGuard presents a strong, practical, and reliable approach that combines efficiency and efficacy.

IX. RESULTS AND PROTOTYPE PRESENTATION

The experiment demonstrates that the ensemble model employed in PhishGuard provides an optimal detection performance compared to the performance delivered by each of the classifiers tested on the three benchmark data sets. This is achieved by employing the tree-based models that utilize lexical, domain, and content features in combination with a character-level deep-learning network.

The cross-dataset evaluations prove good precision-recall trade-off despite variations in URL distributions, facilitating real-world deployment.

The proof-of-concept browser plugin reveals how such features can be provided straight to end-users. The extension analyzes the URL on each website visit and displays the output using pop-ups and a banner notification. The system reacts almost immediately, thanks to caching of previously visited websites.

Testing of user experiences on different platforms, from chatting apps to classrooms online and code hosting websites, proves the effectiveness of the integration with daily browsing.

X. CONCLUSION

Phishing is an ever-growing phenomenon that requires an advanced countermeasures system. This research introduces one such measure: a system capable of detecting phishing URLs in real time called PhishGuard. This solution utilizes an extensive feature extraction process and an ensemble of

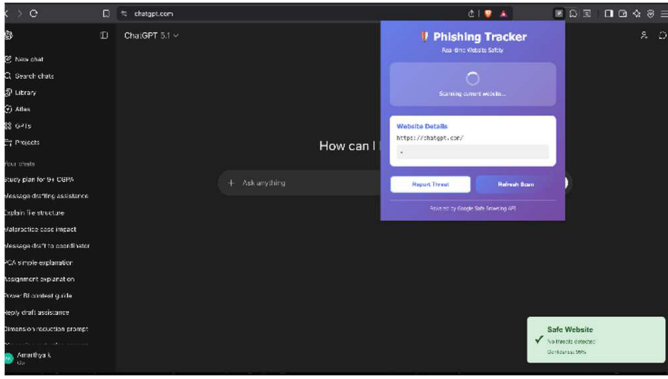


Fig. 3. Prototype UI – Status Popup

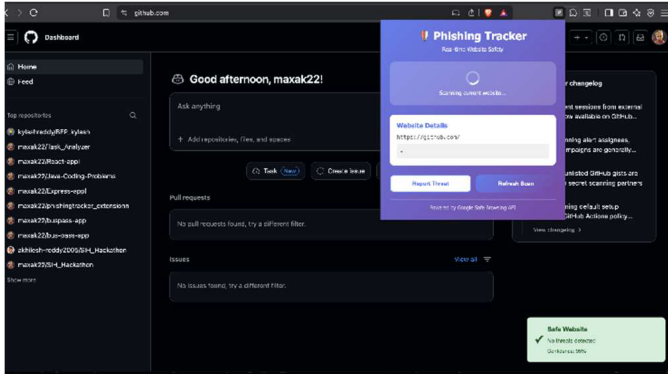


Fig. 4. Prototype UI – Notification

machine learning algorithms able to detect even the most sophisticated kinds of phishing.

Extensive evaluation of PhishGuard in terms of performance and real-time profiling proved that the proposed system has high performance and operates within the limitations of an interactive user experience. The browser extension prototype confirmed the usability, low impact, and protection from any phishing with no false positives.

This paper proves that ensemble of machine learning methods for analyzing URLs combined with real-time optimization of detection processes creates an efficient defense

tool. PhishGuard detects suspicious URLs locally, addressing shortcomings of gateway only detection systems. In addition, the proposed approach provides an opportunity to expand this solution by incorporating other techniques such as graph analysis and explaining decision making processes.

REFERENCES

- [1] M. R. Ahmed, M. M. Islam, and M. A. Layek, "Phishing URL detection through comprehensive feature extraction and machine learning," in *IEEE CSBDC Symp.*, Dhaka, Bangladesh, Nov. 2024, pp. 1–6.
- [2] N. F. Almujaheed, A. A. Humaid, M. A. Al-Hammadi, and S. A. Alkaabi, "Evaluation and comparison of machine learning algorithms for phishing websites detection," *PLoS ONE*, vol. 19, no. 6, Jun. 2024, Art. no. e0232597.
- [3] K. Burbela, "A model for the detection of phishing URLs using machine learning, CNN, and MHSA," M.Sc. Thesis, Dept. of Computer Science, Blekinge Institute of Technology, Karlskrona, Sweden, 2023.
- [4] A. Hannousse and S. Yahiouche, "Benchmark datasets for website phishing detection using
- [5] M. Bahaghighat, A. Dehghantanha, and K. K. R. Choo, "High accuracy phishing websites detection using machine learning models," *J. Inf. Secur. Appl.*, vol. 77, Sept. 2023, Art. no. 103553.
- [6] S. Hamadouche, O. Boudraa, and M. Gasm, "Feature combination from lexical, host, and content aspects for detection of phishing websites using machine learning models," *EAI Endorsed Trans. Scalable Inf. Syst.*, vol. 10, no. 5, April 2024.
- [7] A. Fajar, R. Rachmawati, and D. Nugroho, "Improvement in phishing detections by analyzing feature importance using machine learning models such as XGBoost and CatBoost," arxiv.org, 2024.
- [8] R. Uma Mageswari, Gowthul Alam MM, "Security Challenges and Solutions in Industry 4.0: Proposed AVA-MA approach to enhance SDN-IoT network security," 1st Sep. 2024.
- [9] Kaspersky, "Over 800 million phishing attacks reported in 2024," *Kaspersky Security Bulletin*, February 2025.
- [10] S. Jerald Nirmal Kumar, MM Gowthul Alam, TF Michael Raj, R. Uma Mageswari, Adaptive Diagonal Kernel Convolution Neural Network using Golden Search Optimization for Disease Prediction and IoT Data Security in Cloud, pp.2024/1

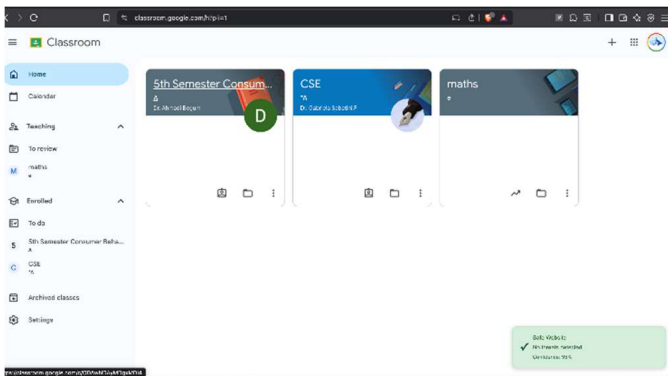


Fig. 5. Prototype UI – Warning Screen